

Experiment-5

Name: S.Sahitya

Roll No: 24501A05M1

Date: 09/09/2026

Aim:Apply the Naïve Bayes classification algorithm on textual or categorical datasets and analyze its robustness using performance metrics like confusion matrix, precision, recall, and accuracy.

Importing Libraries

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_score, confusion_matrix
```

```
# Step 1: Create dataset
data = {
    'age': ['<=30', '<=30', '31...40', '>40', '>40', '>40', '31...40', '<=30', '<=30', '>40', '<=30', '31...40', '31...40', '>40',
    'income': ['high', 'high', 'high', 'medium', 'low', 'low', 'low', 'medium', 'low', 'medium', 'medium', 'medium', 'high', 'medium',
    'student': ['no', 'no', 'no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'yes', 'no', 'no', 'yes', 'no'],
    'credit_rating': ['fair', 'excellent', 'fair', 'fair', 'fair', 'excellent', 'excellent', 'fair', 'fair', 'fair', 'fair', 'excellent',
    'buys_computer': ['no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'yes', 'no']
}
```

```
df = pd.DataFrame(data)
```

```
# Step 2: Encode categorical columns
encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le
```

```
# Features and Target
X = df.drop('buys_computer', axis=1)
y = df['buys_computer']
```

X

	age	income	student	credit_rating
0	1	0	0	1
1	1	0	0	0
2	0	0	0	1
3	2	2	0	1
4	2	1	1	1
5	2	1	1	0
6	0	1	1	0
7	1	2	0	1
8	1	1	1	1
9	2	2	1	1
10	1	2	0	0
11	0	2	0	0
12	0	0	1	1
13	2	2	0	0

y

buys_computer	
0	0
1	0
2	1
3	1
4	1
5	0
6	1
7	0
8	1
9	1
10	1
11	1
12	1
13	0

dtype: int64

```
# Split data into training and testing sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Step 3: Train Naive Bayes
model = CategoricalNB()
model.fit(X_train, y_train)
```

▼ CategoricalNB ⓘ ?

CategoricalNB()

```
# Make predictions
y_pred = model.predict(X_test)
# Evaluate the model
print("confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```

confusion Matrix:

```
[[1 1]
 [0 3]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.50	0.67	2
1	0.75	1.00	0.86	3
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5

Accuracy: 0.8
Precision: 0.75
Recall: 1.0

y_test

buys_computer

9	1
11	1
0	0
12	1
5	0

dtype: int64

y_pred

array([1, 1, 0, 1, 1])

```
# Step 4: Take user input
print("Enter details for prediction:")
age = input("Age (<=30, 31...40, >40): ")
income = input("Income (high, medium, low): ")
student = input("Student (yes, no): ")
credit_rating = input("Credit Rating (fair, excellent): ")
```

```
Enter details for prediction:
Age (<=30, 31...40, >40): >40
Income (high, medium, low): low
Student (yes, no): yes
Credit Rating (fair, excellent): excellent
```

```
# Encode user input
is_valid_input = True
try:
    age_enc = encoders['age'].transform([age])[0]
    income_enc = encoders['income'].transform([income])[0]
    student_enc = encoders['student'].transform([student])[0]
    credit_enc = encoders['credit_rating'].transform([credit_rating])[0]
except ValueError as e:
    print(f"Error encoding input: {e}. Please enter valid category values.")
    is_valid_input = False
```

```
print(age_enc)
print(income_enc)
print(student_enc)
print(credit_enc)
```

```
2
1
1
0
```

```
sample = [[age_enc, income_enc, student_enc, credit_enc]]
```

```
# Step 5: Predict
pred_class = model.predict(sample)[0]
pred_prob = model.predict_proba(sample)[0]

class_label = encoders['buys_computer'].inverse_transform([pred_class])[0]
print(f"\nPredicted Class: {class_label}")
print(f"Class Probabilities: {pred_prob}")
```

```
Predicted Class: yes
Class Probabilities: [0.10714286 0.89285714]
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

